

## 4. Software Solutions: Dekker's Algorithm

**Syllabus Coverage:** Gateway Classes Slides 108–120 | 1st Correct Software Mutual Exclusion Algorithm, Collision Resolution, Comparison with Peterson.

### 4.1 Core Code & Logic

```
flag[i] = true;
while (flag[j]) {
    if (turn == j) {
        flag[i] = false;    // Polite back-off to prevent Livelock
        while (turn == j);  // Wait until Pj exits CS
        flag[i] = true;     // Re-assert intention
    }
}
/* CRITICAL SECTION */
turn = j;
flag[i] = false;
```

#### Dekker's Algorithm: First Software Mutual Exclusion

Contention Back-off Loop & Tie-Breaker Turn Mechanism

##### Process Pi (Pi & Pj Contention Flow)

```
flag[i] = true; // 1. Declare intent
while (flag[j]) { // 2. Is Pj interested?
    if (turn == j) { // 3. If turn belongs to Pj:
        flag[i] = false; // Pi backs off (yields)
        while (turn == j); // Wait until Pj finishes
        flag[i] = true; // Pi re-asserts intent
    }
}
/* CRITICAL SECTION */
turn = j; // Give next turn to Pj
flag[i] = false; // Release intention
/* Remainder Section */
```

##### Dekker's vs Peterson's Comparison

###### 1. Complexity & Structure

Dekker requires nested while loops & explicit back-off. Peterson is a single elegant while loop condition.

###### 2. Intent Withdrawal (Yielding)

Dekker temporarily sets `flag[i]=false` to prevent livelock. Peterson never resets flag until exiting CS completely.

###### 3. Historical Context

Dekker (1965) was the 1st correct software algorithm. Peterson (1981) simplified it 16 years later.

###### Mutual Exclusion: ✓ YES

Dono ek saath CS mein kabhi nahi ghus sakte kyunki turn collision break karta hai.

###### Progress: ✓ YES

Inactive process dusre ko nahi rokta. Livelock back-off logic se resolve hota hai.

###### Bounded Waiting: ✓ YES

Exiting process explicitly `turn = j` pass karta hai, preventing indefinite starvation.

###### Limitation: Busy Waiting

Both while loops consume continuous CPU cycles (Spinlocks).

### 4.2 Comparison Table: Dekker vs Peterson

| Feature                   | Dekker's Algorithm (1965)                        | Peterson's Algorithm (1981)                       |
|---------------------------|--------------------------------------------------|---------------------------------------------------|
| <b>Structure</b>          | Nested loops with explicit back-off.             | Single concise while condition.                   |
| <b>Intention Reset</b>    | Temporarily drops flag to false during conflict. | Keeps flag true; modifies <code>turn</code> only. |
| <b>Criteria Satisfied</b> | Mutual Exclusion, Progress, Bounded Waiting.     | Mutual Exclusion, Progress, Bounded Waiting.      |
| <b>Instruction Count</b>  | Higher (more branches and checks).               | Lower (minimal overhead).                         |